

리눅스 명령어 & 패키지 정리

grep · fdfind (fd) · fdupes · rdfind

실전 예제와 옵션, 비교표로 보는 리눅스 검색·중복파일 도구

1

grep

파일/스트림에서 패턴을 검색하는 기본 텍스트 검색 도구

grep — 기본 문법과 검색

grep은 파일이나 입력 스트림에서 특정 패턴(문자열, 정규표현식)을 검색하는 명령어입니다.

기본 문법

```
grep [옵션] "검색패턴" 파일명
```

기본 검색 예시

```
# 파일에서 "error" 문자열이 포함된 줄 찾기
grep "error" app.log
```

```
# 예시 출력
# 2026-08-11 10:23:01 error: connection timeout
# 2026-08-11 10:25:44 error: invalid token
```

grep — 자주 쓰는 옵션 ①

옵션	설명
-i	대소문자 구분 없이 검색 (ERROR, Error, error 모두 매칭)
-n	일치하는 줄의 번호 표시
-r / -R	디렉터리 재귀 검색
-v	패턴이 없는 줄만 출력 (반전 검색)
-c	일치하는 줄의 개수만 출력
-l / -L	패턴이 있는/없는 파일명만 출력
-w	단어 단위로 정확히 일치하는 경우만 검색

grep — 앞뒤 문맥 출력 (-A, -B, -C)

일치한 줄 뒤로 3줄 더 출력 (After)

```
grep -A 3 "Traceback" error.log
```

일치한 줄 앞으로 2줄 출력 (Before)

```
grep -B 2 "Traceback" error.log
```

앞뒤로 각각 2줄씩 출력 (Context)

```
grep -C 2 "Traceback" error.log
```

-A(After) / -B(Before) / -C(Context) 는 로그 분석 시 원인 파악에 매우 유용합니다.

grep — 정규표현식 활용 (-E)

```
# "error" 또는 "warning"이 포함된 줄  
grep -E "error|warning" app.log
```

```
# 이메일 형식 찾기  
grep -E "[a-zA-Z0-9._%+- ]+@[a-zA-Z0-9.- ]+\.[a-zA-Z]{2,}" contacts.txt
```

```
# 숫자로 시작하는 줄  
grep -E "^[0-9]" data.txt
```

```
# IP 주소 패턴 찾기  
grep -E "([0-9]{1,3}\.){3}[0-9]{1,3}" access.log
```

grep — 실전 활용 예제

1) 특정 프로세스 찾기

```
ps aux | grep "python"
```

2) 여러 파일 확장자에서 검색

```
grep -r --include="*.py" "def calculate" ./project
```

3) 특정 패턴 제외하고 검색

```
grep -r --exclude="*.log" "TODO" ./project
```

4) 여러 패턴을 파일로 관리해서 검색

```
grep -f patterns.txt data.txt
```

5) CSV 파일에서 특정 컬럼 값 찾기

```
grep "FAIL" diagnostic_data.csv | head -20
```

6) git 커밋 이력에서 검색

```
git log -p | grep -n "deprecated_function"
```

grep — 옵션 조합 & 종류 비교

```
# 대소문자 무시 + 줄번호 + 재귀 검색
grep -rin "connection refused" ./logs
```

```
# 정확한 단어 매칭 + 앞뒤 문맥
grep -w -C 2 "timeout" server.log
```

명령어	설명
grep	기본 정규표현식(BRE)
grep -E (= egrep)	확장 정규표현식(ERE), , +, ? 사용
grep -F (= fgrep)	정규식 아닌 순수 문자열 검색 (속도 빠름)

2

fdfind (fd)

find의 대체 도구 — 더 빠르고 직관적인 파일 검색

fdfind — 개요 및 기본 문법

fdfind(fd)는 find의 대체 도구로, 더 빠르고 직관적인 문법을 가진 파일/디렉터리 검색 프로그램입니다.

Ubuntu/Debian에서는 이름 충돌로 fdfind로 설치됩니다 (alias fd='fdfind' 로 별칭 설정 가능).

```
fdfind [옵션] [패턴] [검색경로]
```

```
# 기본 검색 (현재 디렉터리에서 "config" 포함 파일/디렉터리)
```

```
fdfind config
```

grep과 달리 기본적으로 .gitignore, 숨김 파일, node_modules 등을 자동으로 제외합니다.

fdfind — 자주 쓰는 옵션

옵션	설명
-e (--extension)	확장자로 검색 (여러 번 지정 가능)
-t (--type)	타입 지정: f(파일) d(디렉터리) l(심볼릭링크) x(실행파일)
-H (--hidden)	숨김 파일 포함 검색
-I (--no-ignore)	.gitignore 규칙 무시하고 전체 검색
-i / -s	대소문자 무시 / 대소문자 구분 강제
-d (--max-depth)	검색 깊이 제한
-x / -X	찾은 결과 각각에 / 전체에 명령 실행
-g (--glob)	정규식 대신 글롭(와일드카드) 패턴 사용

fdfind — 실전 활용 예제

1) 확장자 필터 + 크기순 정렬

```
fdfind -e csv -x ls -lh {} \; | sort -k5 -h
```

2) 대용량 CSV 파일 찾기 (예: 진단 데이터)

```
fdfind -e csv "diagnostic" ./data
```

3) 특정 디렉터리 제외

```
fdfind -E "node_modules" -E "__pycache__" -e py
```

4) 최근 변경된 파일

```
fdfind --changed-within 1d
```

```
fdfind --changed-within 2weeks
```

5) 파일 크기로 필터링 (10MB 이상 / 1KB 미만)

```
fdfind -S +10M
```

```
fdfind -S -1k
```

6) grep과 파이프 연결

```
fdfind -e py -X grep -l "import torch"
```

fdfind — 옵션 조합 & find 대비 장점

```
# 숨김 파일 포함 + gitignore 무시 + 확장자 + 제외 디렉터리
fdfind -H -I -e py -E venv -E __pycache__ "test_"
```

```
# 최근 수정된 설정 파일 백업
fdfind -e yaml -e json --changed-within 1d -x cp {} ./backup/
```

항목	find	fdfind
검색 속도	느림	병렬 처리로 빠름
문법	복잡(-name, -iname 등)	직관적
기본 제외 규칙	없음	.gitignore·숨김파일 자동 제외
정규표현식	기본 미지원	기본 지원

3

fdupes

중복 파일을 찾아 정리하는 도구

fdupes — 개요, 설치, 기본 사용

fdupes는 중복 파일을 찾아주는 도구입니다. 중복 그룹별로 결과를 표준 출력으로 보여줍니다.

설치

```
sudo apt install fdupes
```

기본 문법

```
fdupes [옵션] 디렉터리
```

기본 사용 (현재 디렉터리, 비재귀)

```
fdupes .
```

예시 출력 (중복 그룹은 빈 줄로 구분)

```
./photos/image1.jpg
```

```
./backup/image1_copy.jpg
```

fdupes — 자주 쓰는 옵션

옵션	설명
-r (--recurse)	하위 디렉터리까지 재귀 검색
-R	지정 인자 이후 디렉터리만 재귀 검색
-n (--noempty)	빈 파일(0바이트) 제외
-s (--symlinks)	심볼릭 링크도 검사 대상에 포함
-q (--quiet)	진행 상태 메시지 숨김
-f (--omitfirst)	각 그룹의 첫 번째 파일은 목록에서 제외

-d 옵션은 대화형 삭제로, 실수로 원본까지 삭제될 수 있으니 반드시 백업 후 사용하세요.

fdupes — 실전 활용 예제

1) 중복 파일 개수/용량 요약

```
fdupes -r -m ./data
```

→ 128 duplicate files (in 42 sets), occupying 3.4 megabytes

2) 중복 파일을 대화형으로 골라서 삭제

```
fdupes -r -d ./data
```

3) 첫 번째 파일 자동 보존, 나머지 자동 삭제

```
fdupes -r -d -N ./data
```

4) 중복 목록만 파일로 저장 (삭제 없음)

```
fdupes -rf ./data > duplicates_to_review.txt
```

5) 크기순 정렬 출력

```
fdupes -r --order=size ./data
```

6) 하드링크로 변환 (삭제 없이 용량 절약)

```
fdupes -r -L ./data
```

4

rdfind

해시 비교 기반의 중복 파일 검색·정리 도구

rdfind — 개요, 설치, 기본 사용

rdfind 역시 중복 파일을 찾아주는 도구이며, 기본적으로 재귀 검색이고 여러 디렉터리를 동시에 지정할 수 있습니다.

설치

```
sudo apt install rdfind
```

기본 문법

```
rdfind [옵션] 디렉터리1 [디렉터리2 ...]
```

기본 사용 (결과는 results.txt 로 저장됨)

```
rdfind ./data
```

```
cat results.txt
```

rfind — 자주 쓰는 옵션

옵션	설명
-dryrun true	실제 변경 없이 시뮬레이션만 수행
-removeidentinode	동일 inode(하드링크) 파일을 중복으로 취급할지 여부
-ignoreempty true	빈 파일 무시
-minsize N	최소 크기(바이트) 이하 파일 제외
-deleteduplicates true	중복 파일 실제 삭제
-makesymlinks true	중복 파일을 심볼릭 링크로 치환
-makehardlinks true	중복 파일을 하드링크로 치환 (용량 절약)
-outputname 파일명	결과 파일명 지정

rdfind — 우선순위 지정 & 실전 예제

여러 디렉터리를 순서대로 지정하면, 먼저 지정한 디렉터리의 파일을 원본(우선순위 높음)으로 취급합니다.

```
# originals를 원본으로, backup의 동일 파일을 중복으로 판단
rdfind ./originals ./backup
```

```
# 1) 안전하게 먼저 결과만 확인
rdfind -dryrun true ./data && cat results.txt
```

```
# 2) 결과 확인 후 실제 삭제
rdfind -deleteduplicates true ./data
```

```
# 3) 백업 폴더 용량 절약을 위한 하드링크 전환
rdfind -makehardlinks true ./backup_2025 ./backup_2026
```

```
# 4) 큰 파일 위주 검사 (예: 사진/동영상)
rdfind -minsize 1000000 ./photos
```

fdupes vs rfind — 비교

항목	fdupes	rfind
결과 확인 방식	표준 출력(그룹별)	results.txt 파일로 저장
삭제 방식	대화형(-d) / 자동(-N)	옵션 일괄 삭제(-deleteduplicates)
하드링크 변환	지원 (-L)	지원 (-makehardlinks)
심볼릭링크 변환	미지원	지원 (-makesymlinks)
원본 우선순위 지정	제한적	디렉터리 순서로 명확히 지정 가능
dry-run 시뮬레이션	없음	지원 (-dryrun true)
대용량 처리 성능	준수	해시 비교 방식, 대용량에 유리

정리

실무 추천 워크플로우

```
# 1단계: rdfind로 먼저 안전하게 시뮬레이션  
rdfind -dryrun true ./data
```

```
# 2단계: results.txt 검토  
less results.txt
```

```
# 3단계: 문제 없으면 fdupes로 대화형 삭제 (더 세밀한 선택)  
fdupes -rd ./data
```

두 도구 모두 삭제 옵션은 되돌릴 수 없으니, 실행 전 `-dryrun` 또는 목록 출력으로 반드시 먼저 확인하세요.